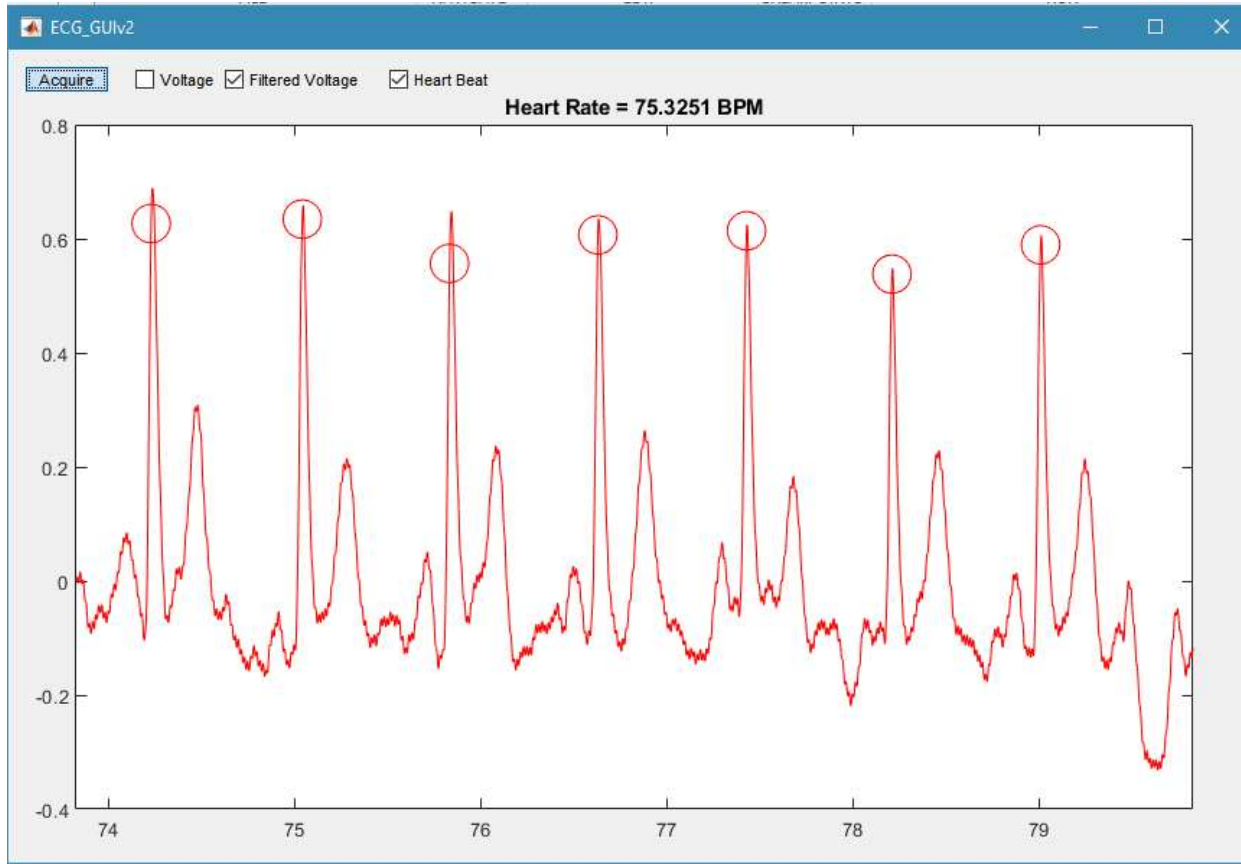




In this exercise we will combine our experience in signal processing, data acquisition, and GUI development. I have created a simulated signal that is an ECG recording using the same circuit you constructed in class. This will allow you to develop the GUI without being connected to the ECG circuit. Once the algorithm is working, then you can connect to the myDAQ and circuit to perform the live data acquisition. **You are welcome to use AI/LLM tools to assist with this assignment.**

- 1) **Create a new GUI in the appdesigner.** Add a UIAxes object (for displaying the ECG signal) and a state button (aka a toggle button, for starting and stopping acquisition); change the button text to something descriptive (like 'Acquire Data'). A state (toggle) button will stay on (value = 1) when clicked once; a second click will turn the button off (value = 0). Run the GUI to make sure there are no errors; you should see a plot window and a toggle button that does nothing.
- 2) **Download the Files from CANVAS.** There are a set of files on CANVAS. Download these files and save them into the same directory as your GUI. The following files are included (see the video for details on functionality):
 - a. **DAQ_ECG_Listener.m** is the template for the listener function that is called whenever the DAQ (or simulated DAQ) has data available. You will edit this to perform the beat detection and do the plots.
 - b. **setupTimeObj.m** is a script that initializes the simulated DAQ session, or the myDAQ if that is connected. Do not edit this function.
 - c. **simNI_DAQ.mat** is a datafile that contains the ECG signal in a data structure. Do not edit.
 - d. **stopFcn.m** is called when the simulated DAQ session is stopped. Do not edit.
 - e. **timerFxnSimNI.m** is the function that simulates the DAQ recording the ECG signal. Do not edit.

3) **Define app properties and the startupFcn callback.** First, define the following public properties:

```
% Flag to indicate if the GUI should try to connect to the myDAQ

TryToUseRealDAQ = 0; %ONLY SET TO 1 if you have the drivers installed!
sampleRate = 512; %DAQ sample rate
sesh % DAQ session object (real or simulated)
filterObj %filter object for processing incoming signal
winSize = 6; %width of plot window, in seconds

%data
time = []; %time vector from DAQ
data = []; %aquired raw voltage data

simNI_DAQ %simulated DAQ object
```

The provided script (setupTimeObj.m) sets up the DAQ (or simulated DAQ) for **continuous background acquisition**. When you start the data acquisition session, the DAQ will run continuously until stopped. This execution will take place in the background; other MATLAB operations can be performed while the background acquisition is occurring.

In the startupFcn:

- a) Call the setupTimeObj script that you downloaded previously. If app.TryToUseRealDAQ == 1, this will try to connect to the myDAQ. Otherwise, or if the myDAQ is not connected, it will create a timer object that simulates the myDAQ and ECG circuit.
- b) Define a filter object and store it in filterObj public property. The filter is a bandpass infinite impulse response; f1 = 0.1; f2 = 20; and 4th order. This is the same as the other ECG analysis assignments, you're encouraged to copy your old code.

4) **Define the Acquisition Callback.** This callback is executed whenever the 'Acquire' button is clicked. When the button is depressed (and the value set to one), the recording is started. When the button is released (value set to zero) the recording is stopped. Go to the ValueChanged callback function for your toggle button and add these commands (in the presented order!):

- a. If the button value is 1 (button is depressed)
 - i. Set the data and time properties to be empty vectors []. This is where the recorded data will be stored, and we want to clear out any old data.
 - ii. Add an 'if' statement so that 'app.sesh.startBackground()' is executed if 'app.sesh.IsRunning' is equal to zero. This will start a background acquisition if an acquisition is not currently running. Note that the empty parentheses are critical.
- b. If the button value is 0 (button is not depressed) then we stop the acquisition with
 - i. app.sesh.stop();
 - ii. Note that you must include the parentheses

Run your GUI and toggle doAcquire on and off. Nothing should happen (because we have not implemented the listener/plotting), and no errors should result.

5) **The listener.** The listener function is called automatically whenever data is available on the DAQ (or simulated DAQ). This occurs every 1/10th second with the default settings. The input arguments to the listener function are detailed in the function template 'DAQ_ECG_Listener.m' (provided on CANVAS). This template has numbered sections where you'll fill in your code.

- a. First, to test if the listener is being called correctly, plot the 1/10th second blocs of data as they come into the function. This will not look nice since we're only plotting 1/10 sec at a time, but it will show you if the function is getting called. Add this command to the function template:

```
plot(app.UIAxes, event.TimeStamps,event.Data)
```

Then run the app, and click the acquire button to test. If the plotting fails, or you get errors/warnings, debug those now!

- b. Once you have confirmed the listener is being called, delete the previously specified plot command. To accumulate data, concatenate that DAQ timestamps into the time property of the app. Do the same for the DAQ data (event.Data) and the data property of the app. This is section 1 in the template.

```
app.time = [app.time event.TimeStamps'];
```

- c. Plot app.time and app.data on axes1. Run the acquisition. You should see the ECG getting longer as time progresses.
- d. We only want to keep data that is inside the plotting window (that has length app.winSize=6, defined in the main GUI code). Delete all elements of app.time and app.data that correspond to times outside the plotting window by first finding the indices of those points; then deleting the values. Run and inspect the plot to confirm this is working. *This is section 2 in the template.*
- e. Our plot looks a little funky because the x-axis limits are not tight to the signal. Set the x-axis limits to be the first and last elements of the time vector.

```
xlim(app.UIAxes,[app.time(1) app.time(end)])
```

- f. Now we'll add your 'HR_Detect' algorithm to the listener. Find that file and copy it to the same directory as your GUI. Since our signal can be as short as 0.1 seconds, we need to modify the peak-finding section to only run when the signal is longer than the minimum peak distance. If we don't do this, you'll get an error because the signal is shorter than the minimum peak distance (0.2 sec).

```
if MPD < length(V) % look for peaks when signal is longer than 0.2 s
    [~, locs] = findpeaks(vInt, 'MinPeakDistance', MPD, ...
        'MinPeakHeight', MPH);
else
    locs = []; % if not looking for peaks set locs to empty
end
```

- g. Once you've modified HR_Detect, it can be added to *section 3 in the listener template*. For the inputs, note that the voltage is app.data, the sample rate is src.Rate, and the filter object is app.filterObj (defined in the opening function of your GUI). Run your GUI to test for errors.

Note that error messages can be confusing since they are occurring inside the timer function. I STRONGLY recommend setting a breakpoint in the listener function (before any errors!), running to that breakpoint, and then evaluating your code line-by-line to make sure it is error-free.

- 6) **Plotting.** Now we'll upgrade the plot sections. In the appdesigner, create three checkboxes that will be used to control plotting. Activating any of these three checkboxes will result in the associated plot being generated when acquiring data.
 - a. 'Voltage', with a default value of 1 (checked)
 - b. 'Filtered', with a default value of 0 (unchecked)
 - c. 'Heart Beat', with a default value of 0 (unchecked)

Save the app, and then open the 'DAQ_ECG_Listener' function. In the plotting section

- a. Turn hold off (on the UIAxes) before any of the plots

- b. Plot the raw voltage if the 'Voltage' checkbox is checked. Turn hold on after the plot (inside the if).
- c. Plot the filtered voltage when the 'Filtered' box is checked. Include a hold on after the plot command (inside the if).
- d. Plot the time of each beat (x-axis) and the value of the filtered voltage for each beat (y-axis) with a red 'o' and a marker size of 20 when the 'Heart Beat' box is checked. Recall you'll need to use locs to access the correct entries of app.time and the filtered voltage.

Test all combinations of the plots.

- 7) **Adding the numeric value for the heart rate.** Recall from the previous assignment that the period of the heart beat is the time between beats. Therefore the vector of periods can be calculated using `'time(locsNow(2:end)) - time(locsNow(1:end-1))'`. The frequency (heart rate in beats per second) is $1/\text{period}$.

Set the title of your plot to be the **mean** heart rate in the plot window in beats per minute. The `num2str` command can be used to convert your calculated mean heart rate (a number) into a string that can be displayed in the plot title. Include some descriptive text describing the number.

- 8) **Clean up on app shutdown.** In the `UIFigure closeRequest` callback (which is called when the app is closed), place the following code:

```
app.sesh.stop(); %stop the data aquisition if it is running
activeTimers = timerfind; %find all running timer functions
if ~isempty(activeTimers) %if there's still a timer running then
    stop(activeTimers); %stop all timers
    delete(activeTimers); %delete all timers
end
delete(app) %delete the app window
```

Note: if you can't close your app (because of errors, etc) then run 'close all force' in the command window.

- 9) **Testing and Demonstration with your ECG circuit.** Retrieve your ECG circuit and connect it to the DAQ. Demonstrate to the TA or instructor
- a. A live plot of raw voltage, heart rate correctly displayed in title
 - b. A live plot of raw voltage, location of beats indicated with red circles
 - c. A live plot of filtered and raw voltages
 - d. A plot of filtered and raw voltages, location of beats indicated with red circles

Submission

Please upload a zip file to CANVAS that contains all of the files necessary to run your GUI. I suggest zipping the entire directory.